

Tecnicatura Universitaria en Inteligencia Artificial

FCEIA — Universidad Nacional de Rosario

Trabajo Práctico N° 1

Redes Neuronales Feed Forward

Integrantes:

Genaro Canciani

Juan Andrés Morales

Nicolás Mancini

Materia: Aprendizaje Automático II

Abril de 2026

Índice

Problema 1: Clasificación Binaria — Riesgo de ACV

1	Introducción	2
2	Análisis Exploratorio de Datos	2
3	Preprocesamiento	3
4	Diseño de la Red Neuronal	3
5	Entrenamiento y Resultados	4
6	Evaluación del Modelo	4

Problema 2: Clasificación Multiclase — MNIST

7	Descripción del Procedimiento Seguido	6
8	Decisiones de Diseño y Justificaciones	6
9	Desafíos Encontrados	7
10	Conclusiones y Aprendizajes Obtenidos	8

Problema 3: Regresión — Dataset Superstore

11	Descripción del Procedimiento Seguido	9
12	Análisis Exploratorio de Datos (EDA)	10
12.1	Exploración de las variables y sus tipos de datos	10
12.2	Distribución de la variable objetivo	10
12.3	Relaciones entre variables numéricas y Profit	10
13	Preprocesamiento	11
13.1	Ingeniería de features temporales	11
13.2	Codificación de variables	11
13.3	Escalado de variables numéricas	11
14	Diseño e Implementación de la Red Neuronal	11
14.1	Arquitectura del modelo	11
14.2	Entrenamiento y comparación de funciones de pérdida	11
15	Evaluación del Modelo	12
15.1	Métricas sobre el conjunto de test	12
15.2	Análisis de residuos	12
16	Desafíos Encontrados	13
17	Conclusiones y Aprendizajes Obtenidos	13

Problema 1: Clasificación Binaria

Predicción de Riesgo de Accidente Cerebrovascular (ACV)

Introducción

El objetivo de este módulo es diseñar y evaluar una red neuronal multicapa (MLP) para predecir el riesgo de accidente cerebrovascular (ACV) a partir de datos clínicos y demográficos. El problema se define como una clasificación binaria sobre un dataset de 5110 pacientes. La naturaleza del problema exige una priorización absoluta de la detección de casos positivos, dado que omitir a un paciente en riesgo conlleva consecuencias clínicas graves.

Análisis Exploratorio de Datos

El dataset presenta un desbalance de clases crítico. El 95.1 % de los registros pertenecen a la clase negativa (Sin ACV), mientras que solo el 4.9 % corresponden a la clase positiva. Este factor condiciona toda la estrategia de modelado, invalidando la exactitud (*accuracy*) como métrica de éxito, ya que un modelo ingenuo alcanzaría un rendimiento alto sin detectar un solo caso real.

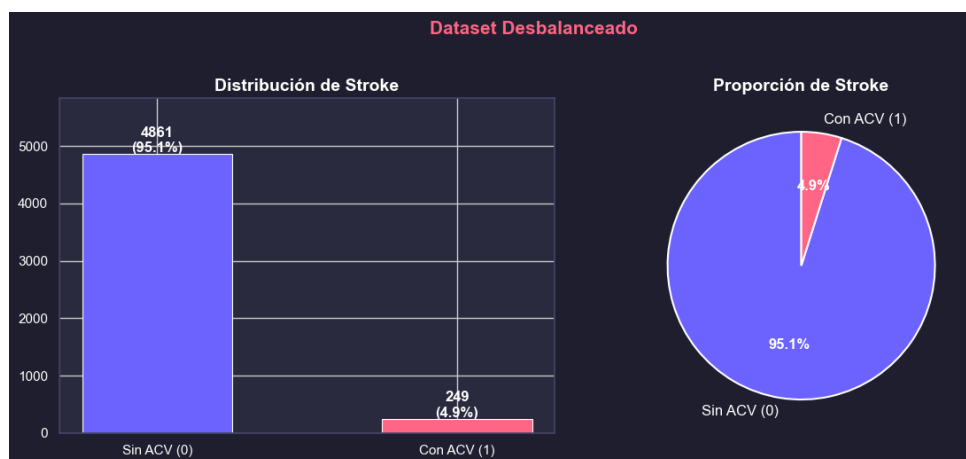


Figura 1: Distribución de la variable objetivo (**stroke**) evidenciando el fuerte desbalance de clases.

A través del análisis de correlación y distribuciones, se identificó la **edad** como el predictor más robusto. La población con ACV muestra un pico de incidencia cerca de los 80 años, mientras que en pacientes sanos la distribución es más uniforme. El nivel de glucosa promedio también mostró un comportamiento bimodal en pacientes con ACV, con un grupo significativo presentando valores superiores a 200 mg/dL.

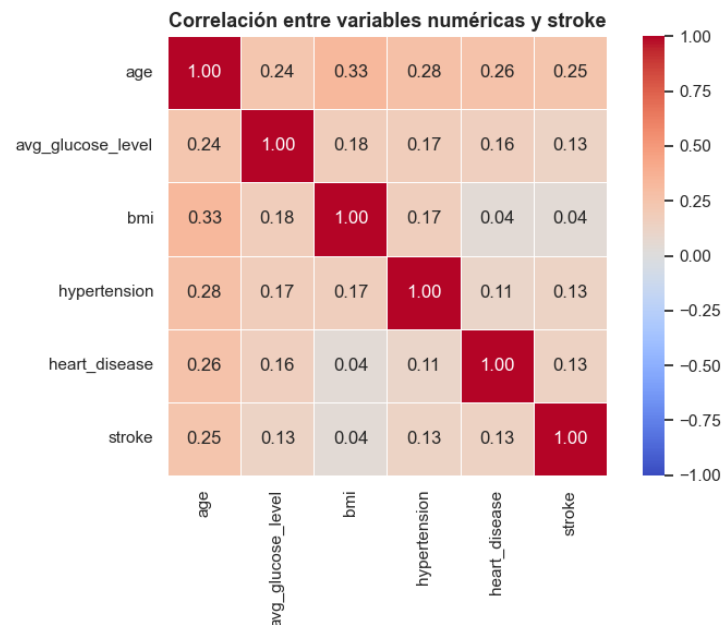


Figura 2: Matriz de correlación entre variables numéricas y la variable objetivo.

El índice de masa corporal (BMI) presentó una correlación lineal nula con la variable objetivo, aunque se detectaron outliers extremos con valores superiores a 60 que requerían limpieza para no sesgar el entrenamiento.

Preprocesamiento

La preparación de los datos se realizó bajo una política estricta de prevención de fuga de información (*data leakage*). Se tomaron las siguientes decisiones:

- Se eliminó el identificador único del paciente por carecer de valor predictivo.
- Los valores de BMI superiores a 60 se trataron como datos erróneos y se reemplazaron por nulos para su posterior imputación.
- Se aplicó un **KNN Imputer** con 5 vecinos para completar los faltantes de BMI.
- Las variables categóricas se codificaron mediante **One-Hot Encoding**, eliminando la primera columna para evitar la colinealidad.
- Las variables numéricas se escalaron utilizando **StandardScaler**. El ajuste de estos transformadores se realizó exclusivamente sobre el conjunto de entrenamiento, aplicándolos luego a validación y test.

Diseño de la Red Neuronal

La arquitectura elegida consiste en un MLP con dos capas ocultas de 32 y 16 neuronas respectivamente. Se utilizó la función de activación ReLU por su eficiencia y se implementó *Dropout* con una tasa de 0.2 en cada capa para mitigar el sobreajuste. La inicialización de pesos siguió el esquema de *Kaiming He*.

Dada la asimetría de las clases, se utilizó la función de pérdida **BCEWithLogitsLoss** configurada con un peso posicional calculado como la razón entre la clase mayoritaria y la minoritaria, penalizando con mayor fuerza los errores en la detección de pacientes con ACV. El optimizador **Adam** se configuró con una tasa de aprendizaje de 0.001 y una penalización L2 para forzar la convergencia.

Entrenamiento y Resultados

El entrenamiento se ejecutó con un tamaño de lote de 32. Se implementó **Early Stopping** con una paciencia de 15 épocas, lo que permitió detener el proceso en la época 25 al detectar que la pérdida de validación dejaba de descender.

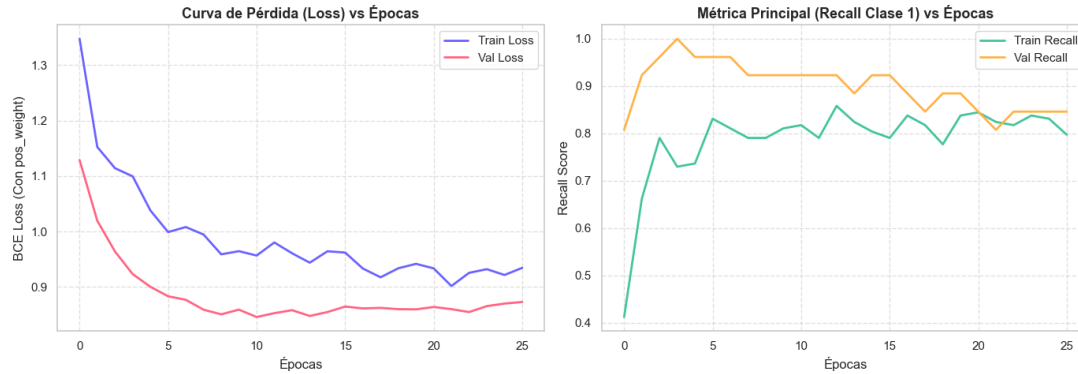


Figura 3: Curvas de pérdida (izq.) y evolución de la métrica Recall (der.) durante el entrenamiento.

Esta técnica garantizó la selección del modelo con mayor capacidad de generalización antes de que la red comenzara a memorizar el ruido del conjunto de entrenamiento. Las curvas de aprendizaje mostraron una convergencia estable. Se alcanzó un Recall estable de aproximadamente el 85 % en validación.

Evaluación del Modelo

La evaluación final en el conjunto de test se realizó optimizando el umbral de decisión. Se buscó el umbral que maximizara el **F2-Score**, otorgando el doble de importancia al Recall que a la Precisión. El umbral óptimo hallado fue **0.47**.

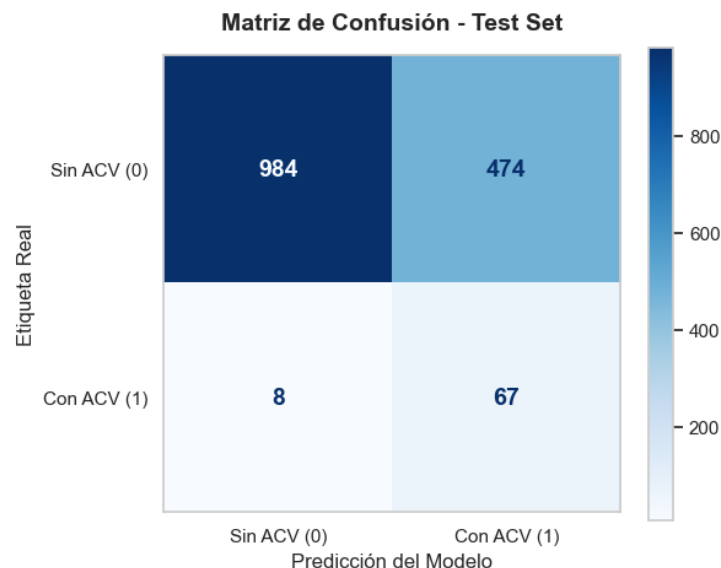


Figura 4: Matriz de confusión sobre el conjunto de test utilizando el umbral optimizado.

Los resultados finales en test arrojaron un **Recall del 89.33 %** y un **AUC-ROC de 0.84**. El modelo identificó correctamente a 67 de los 75 pacientes con ACV real. Si bien la precisión

obtenida es baja, este resultado es aceptable en un contexto de triaje clínico inicial, donde es preferible generar falsas alarmas que dejar de detectar a un paciente en peligro. El sistema se comporta como un filtro de sospecha altamente eficaz que logra reducir el volumen de población a analizar por profesionales médicos, concentrando la atención en el grupo de mayor riesgo real.

Problema 2: Clasificación Multiclase

Reconocimiento de Dígitos Manuscritos — Dataset MNIST

Descripción del Procedimiento Seguido

El problema consistió en desarrollar un modelo de aprendizaje profundo capaz de clasificar imágenes de dígitos escritos a mano (del 0 al 9) pertenecientes al conjunto de datos MNIST. El procedimiento se estructuró en las siguientes fases:

1. **Carga y aplanamiento de datos:** Se leyeron las imágenes directamente desde los archivos binarios, obteniendo matrices de dimensiones (60000, 28, 28) para entrenamiento y (10000, 28, 28) para prueba. Dado que se utilizó un Perceptrón Multicapa (MLP), las imágenes fueron aplanadas a vectores unidimensionales de 784 elementos.
2. **Preprocesamiento y normalización:** Se normalizaron los valores de intensidad de los píxeles, pasando de un rango de [0, 255] a [0, 1]. Las etiquetas se transformaron en tensores de tipo *long* de PyTorch, preparándolas para el cálculo eficiente de la pérdida sin necesidad de un *One-Hot Encoding* explícito.
3. **Definición y entrenamiento del modelo:** Se estructuró una red neuronal profunda y se implementó un bucle de entrenamiento personalizado. Se monitorearon las métricas de pérdida y precisión tanto en el conjunto de entrenamiento como en el de validación, integrando mecanismos de parada temprana (*Early Stopping*).
4. **Evaluación:** Se evaluó el modelo final utilizando el conjunto de prueba, empleando métricas globales e individuales por clase (precisión, recall, F1-score) y herramientas visuales como matrices de confusión y análisis de errores de alta confianza.

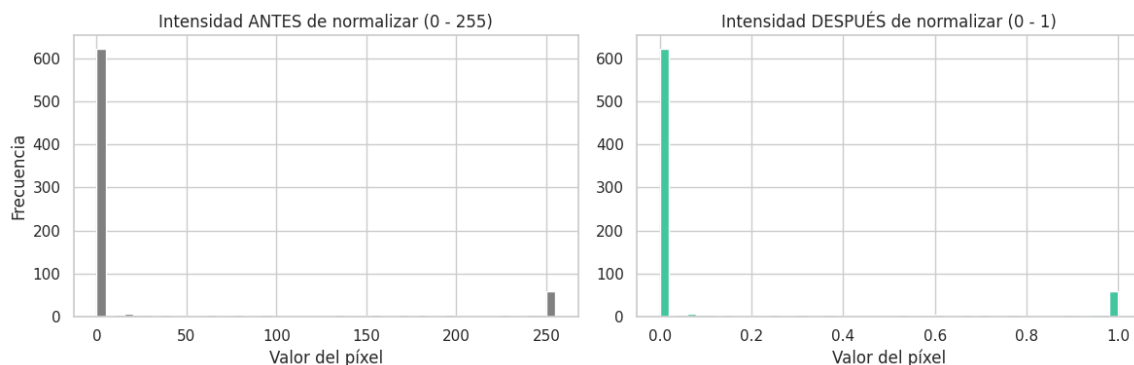


Figura 5: Distribución de la intensidad de los píxeles antes y después de la normalización.

Decisiones de Diseño y Justificaciones

La arquitectura y los hiperparámetros se seleccionaron bajo los siguientes criterios técnicos:

- **Arquitectura (784 → 128 → 64 → 10):** Topología decreciente de dos capas ocultas para compresión jerárquica de datos.
- **Activación e Inicialización:** Uso de *ReLU* e inicialización *He* (*Kaiming Normal*).
- **Capa de Salida y Función de Pérdida:** Logits crudos procesados por *CrossEntropyLoss*.

- **Optimizador:** *Adam* ($lr = 1 \times 10^{-3}$) con *batch size* de 516.
- **Estrategia de Regularización Múltiple:**
 - *Batch Normalization* para estabilidad interna.
 - *Dropout* (30 %) para robustez.
 - *Weight Decay* (1×10^{-4}) y *Early Stopping* (paciencia = 20).

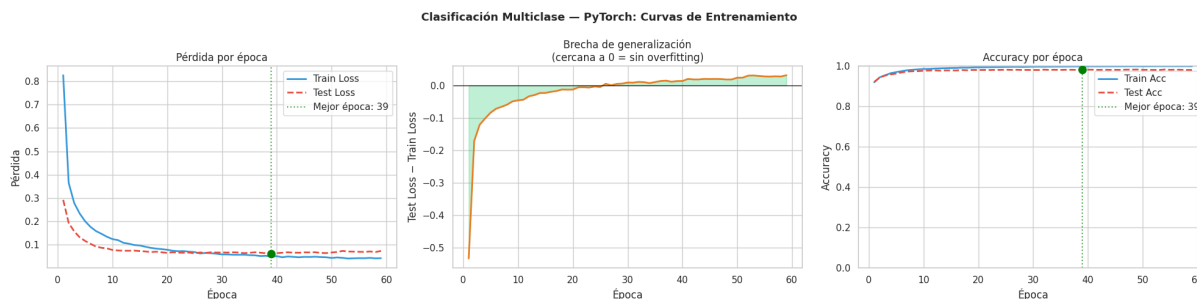


Figura 6: Curvas de pérdida, brecha de generalización y *Accuracy* durante el entrenamiento.

Desafíos Encontrados

El principal desafío radicó en la limitación estructural del modelo para interpretar la geometría. Durante el análisis de errores, el modelo alcanzó un error mínimo, pero se confundió con pares de dígitos visualmente similares (ej. 7 vs. 2, 9 vs. 4) con niveles de confianza superiores al 99 %. Esto ocurre porque ciertos estilos caligráficos comparten patrones locales de píxeles muy fuertes que engañan la extracción de características del MLP.

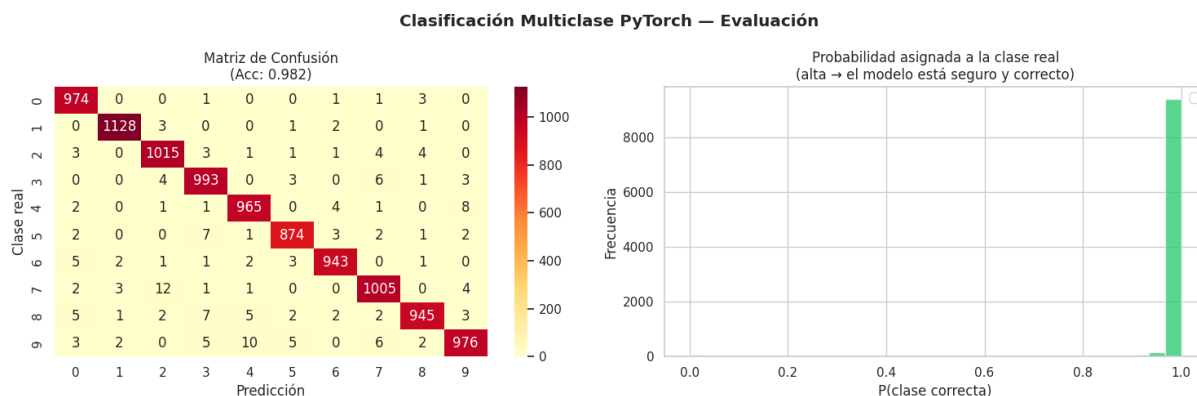


Figura 7: Matriz de confusión y distribución de probabilidades en el conjunto de prueba.



Figura 8: Muestra de imágenes clasificadas incorrectamente por el modelo con alta confianza.

Conclusiones y Aprendizajes Obtenidos

El Perceptrón Multicapa es competente para MNIST (98.18 % de accuracy), pero el aprendizaje más significativo es la identificación de su **ceguera ante estructuras geométricas básicas** debido al aplanamiento de los datos. Esta limitación provoca que el número de parámetros crezca de manera inmanejable ante aumentos de resolución, señalando a las **Redes Neuronales Convolucionales (CNN)** como el paso evolutivo necesario para explotar jerarquías espaciales.

Problema 3: Regresión

Predicción de Rentabilidad (Profit) — Dataset Superstore

Descripción del Procedimiento Seguido

El problema consistió en desarrollar un modelo de aprendizaje profundo capaz de predecir la variable continua **Profit** a partir de un conjunto de features del dataset *Superstore*, que recoge transacciones comerciales de una cadena minorista estadounidense. Al tratarse de una tarea de **regresión**, la salida del modelo es un valor numérico sin cota, lo que lo distingue fundamentalmente de los problemas de clasificación. El procedimiento se estructuró en las siguientes fases:

1. **Carga del dataset:** Se leyó el archivo `superstore.csv` desde disco (formato Latin-1), obteniendo un *DataFrame* de 9994 filas y 21 columnas. Los espacios en los nombres de columnas fueron reemplazados por guiones bajos para facilitar el acceso programático.
2. **Análisis exploratorio (EDA):** Se inspeccionaron los tipos de datos, la presencia de valores nulos, la distribución de la variable objetivo y las relaciones entre las variables numéricas (**Sales**, **Quantity**, **Discount**) y **Profit**.
3. **Preprocesamiento e ingeniería de features:** Se extrajeron variables temporales desde las fechas, se aplicaron codificaciones cíclicas (seno/coseno) para la variable **Month**, y *One-Hot Encoding* para variables categóricas, año, trimestre y mes-año. Las variables numéricas fueron escaladas con **RobustScaler**.
4. **División de datos:** El conjunto fue dividido en 70 % entrenamiento (6995 muestras) y 30 % prueba (2999 muestras) mediante `train_test_split` con semilla fija.
5. **Diseño y entrenamiento de la red neuronal:** Se implementó una red con dos capas ocultas. Se entrenó con tres funciones de pérdida distintas (MSE, MAE y Huber) para comparar su comportamiento, integrando un bucle de entrenamiento con *Early Stopping*.
6. **Evaluación del modelo final:** Se seleccionó el mejor modelo según la pérdida en test, se invirtieron los escalados para obtener predicciones en la escala original del negocio, y se evaluaron las métricas MSE, RMSE y R^2 junto con gráficos de residuos y dispersión.

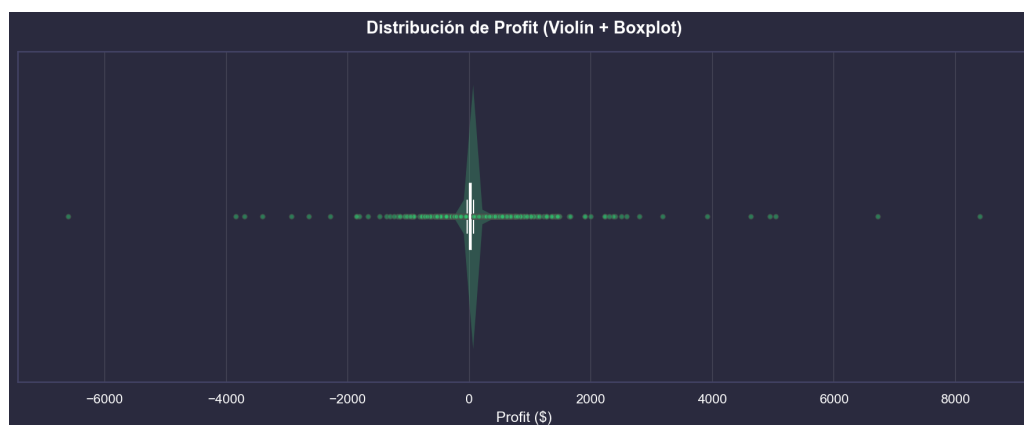


Figura 9: Distribución de la variable objetivo **Profit** (Violín + Boxplot), mostrando la alta concentración en torno a cero y la presencia de valores atípicos.

Análisis Exploratorio de Datos (EDA)

Exploración de las variables y sus tipos de datos

El dataset contiene 21 columnas distribuidas en tres tipos: 3 variables de tipo `float64` (`Sales`, `Discount`, `Profit`), 3 de tipo `int64` (`Row_ID`, `Postal_Code`, `Quantity`) y 15 de tipo `str` (identificadores, fechas, nombres y categorías). No se detectaron valores nulos en ninguna columna, por lo que no fue necesario aplicar estrategias de imputación.

Las variables de mayor relevancia para la predicción son: `Sales` (monto de venta en dólares), `Quantity` (cantidad de unidades vendidas), `Discount` (porcentaje de descuento aplicado, entre 0 y 0.8), `Category` y `Sub-Category` (categorías del producto), `Segment` (segmento del cliente: Consumer, Corporate, Home Office) y `Region` (región geográfica).

Distribución de la variable objetivo

A partir del gráfico de violín con boxplot superpuesto (Figura 9), se observa que la distribución de `Profit` se encuentra centrada cerca de cero, lo que sugiere que muchas órdenes generan ganancias bajas o cercanas al punto de equilibrio. También se detecta la presencia de valores negativos, que representan transacciones con pérdidas, y de *outliers* tanto positivos como negativos que se alejan considerablemente del grueso de los datos. Si bien la mayoría de los valores se concentra en un rango estrecho, la presencia de estos extremos hace que el problema de regresión sea inherentemente desafiante.

Relaciones entre variables numéricas y Profit

Se analizaron gráficamente las relaciones entre `Sales`, `Quantity` y `Discount` respecto a `Profit`:

- **Sales vs. Profit:** Se observa una correlación positiva inicial; a mayores ventas, el potencial de ganancia aumenta. Sin embargo, la dispersión se ensancha significativamente hacia valores altos de `Sales`, confirmando la presencia de outliers que derivan tanto en ganancias excepcionales como en pérdidas considerables.
- **Quantity vs. Profit:** La mayoría de las operaciones se concentra en un rango de beneficios cercano a cero, independientemente del volumen. La distribución es más compacta en comparación con la de ventas, aunque se aprecian puntos aislados de alta rentabilidad o pérdida en casi todos los niveles de cantidad.
- **Discount vs. Profit:** Existe una relación inversamente proporcional clara a partir de ciertos niveles. Descuentos bajos (0 %–20 %) mantienen márgenes positivos, mientras que descuentos hacia el 80 % empujan la mayoría de las observaciones hacia el área de pérdidas, evidenciando un impacto crítico sobre el margen del negocio.

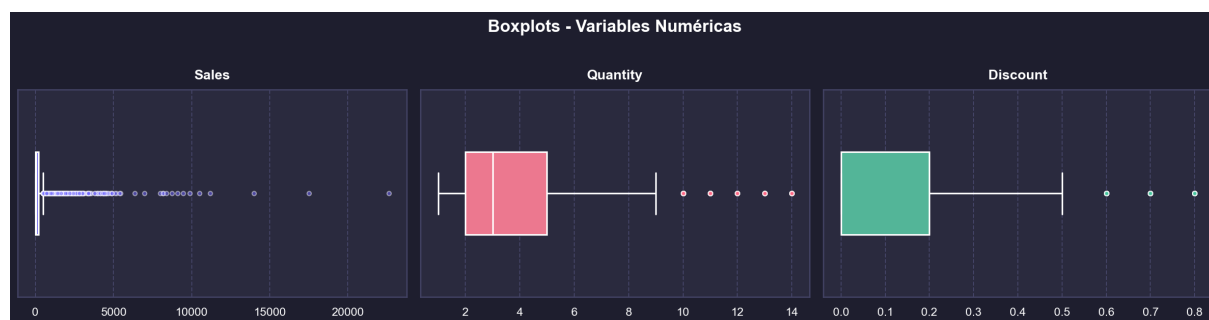


Figura 10: Gráficos de dispersión entre `Sales`, `Quantity` y `Discount` respecto a `Profit`.

Preprocesamiento

Ingeniería de features temporales

Las columnas de fecha (`Order_Date`, `Ship_Date`) fueron convertidas al tipo `datetime` de pandas y se extrajeron las siguientes variables derivadas: `Year`, `Month`, `Quarter` y `YearMonth`. Las columnas de fecha originales y otras columnas no informativas (`Row_ID`, `Order_ID`, `Customer_ID`, `Product_ID`, `Customer_Name`, `Product_Name`, `City`, `State`, `Country`, `Postal_Code`) fueron eliminadas del conjunto de features.

Codificación de variables

Se emplearon dos técnicas de codificación:

- **Codificación cíclica** para `Month`: los valores mensuales se transformaron en coordenadas espaciales mediante funciones de seno y coseno:

$$\text{Month_sin} = \sin\left(\frac{2\pi \cdot \text{Month}}{12}\right), \quad \text{Month_cos} = \cos\left(\frac{2\pi \cdot \text{Month}}{12}\right)$$

Esta representación permite al modelo capturar la estacionalidad circular del calendario.

- **One-Hot Encoding** para las variables categóricas nominales: `Ship_Mode`, `Segment`, `Region`, `Category`, `Sub-Category`, `Year`, `Quarter` y `YearMonth`.

Escalado de variables numéricas

Las variables continuas (`Sales`, `Quantity`, `Discount` y la variable objetivo `Profit`) fueron escaladas utilizando `RobustScaler`. Este escalador se basa en la mediana y el rango intercuartílico (IQR), lo que lo hace significativamente menos sensible a valores extremos que `StandardScaler` o `MinMaxScaler`. El ajuste del escalador se realizó exclusivamente sobre el conjunto de entrenamiento, evitando así la fuga de información.

Diseño e Implementación de la Red Neuronal

Arquitectura del modelo

Se diseñó una red con la siguiente estructura:

- **Capa de entrada:** dimensión igual al número de features tras el preprocesamiento.
- **Dos capas ocultas:** 64 y 32 neuronas respectivamente, con activación *ReLU*.
- **Capa de salida:** 1 neurona lineal (sin función de activación), apropiada para regresión.

Entrenamiento y comparación de funciones de pérdida

Se implementó la función `entrenar_pytorch_regresion`, que ejecuta el ciclo clásico Forward → Loss → Backward → Update con los siguientes hiperparámetros:

Hiperparámetro	Valor
Épocas máximas	300
Batch size	64
Tasa de aprendizaje	10^{-3}
Paciencia (Early Stopping)	30 épocas
Weight Decay	10^{-4}

Se entrenó la misma arquitectura con tres criterios de pérdida distintos para analizar su impacto:

Función de pérdida	Época de parada	Mejor R^2 (test)
MSE (<code>nn.MSELoss</code>)	181	0.9222
MAE (<code>nn.L1Loss</code>)	165	0.8845
Huber (<code>nn.HuberLoss</code> , $\delta = 1$)	160	0.9012

Se seleccionó el **Mean Squared Error (MSE)** como la función de pérdida definitiva, ya que logró el R^2 más alto (92.22 %), superando la robustez teórica de Huber y MAE. Dado que se utilizó **RobustScaler** en el preprocesamiento, el impacto negativo de los outliers ya estaba controlado, permitiendo que el MSE se enfocara en la precisión global.

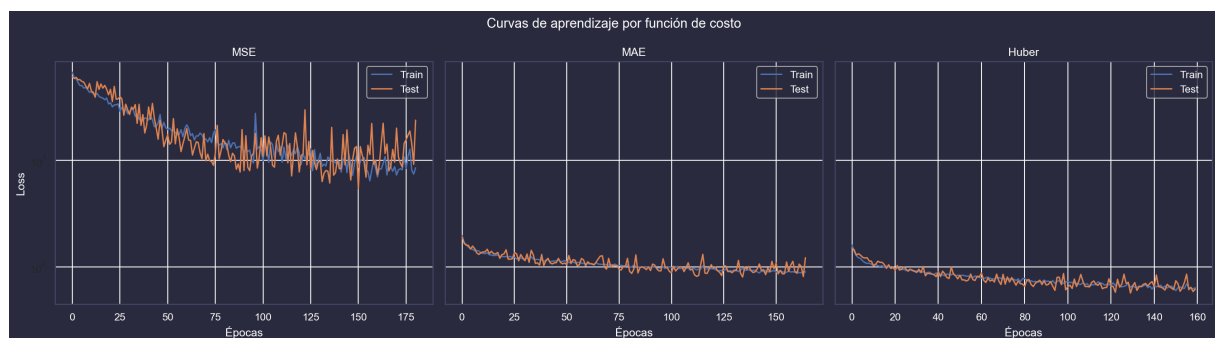


Figura 11: Curvas de pérdida (train y test). El MSE, aunque más volátil inicialmente, logra una convergencia más profunda.

Evaluación del Modelo

Métricas sobre el conjunto de test

Las predicciones fueron revertidas mediante `scaler_y.inverse_transform` para obtener resultados interpretables en la escala del negocio. Las métricas finales sobre el conjunto de prueba son:

Métrica	Valor
MSE	4163.08
RMSE	64.52 (\$)
R^2	0.9222

El modelo alcanzó un $R^2 = 0,9222$, lo que indica que la arquitectura explica más del 92 % de la variabilidad del **Profit**. En términos de negocio, el RMSE implica que las predicciones se desvían aproximadamente ± 64 dólares del valor real. Para transacciones de bajo margen esto puede ser significativo, pero en operaciones de mayor escala representa un error aceptable.

Análisis de residuos

El histograma de residuos presenta una distribución centrada cerca de cero ($\mu = 3,67$, $\sigma = 64,42$), lo que confirma que el modelo **no introduce sesgo sistemático**, es decir, no sobreestima ni subestima de forma constante. Sin embargo, tanto las colas del histograma como la mayor dispersión visible en el gráfico Real vs. Predicho para valores extremos indican que el modelo tiende a cometer errores más grandes cuando el **Profit** es muy alto o muy bajo. Esto

es consecuencia de que dichos casos son poco frecuentes en el dataset, y por ende la red tiene menos oportunidades de aprenderlos correctamente.

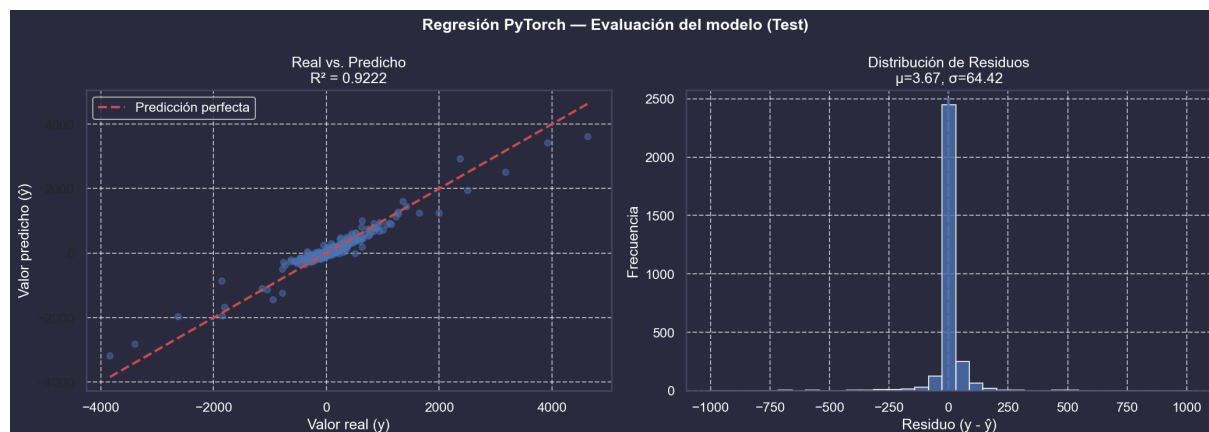


Figura 12: Izquierda: gráfico de dispersión Real vs. Predicho ($R^2 = 0,9222$). Derecha: histograma de residuos ($\mu = 3,67$, $\sigma = 64,42$).

Desafíos Encontrados

El principal desafío radicó en la **alta volatilidad e irregularidad de la variable objetivo**. El **Profit** de una transacción depende de factores interrelacionados (descuento aplicado, categoría del producto, segmento del cliente, temporada) cuyas interacciones son difíciles de capturar con una red completamente conectada que no posee *inductive bias* espacial ni temporal.

Adicionalmente, la presencia de **outliers severos** en **Sales** y **Profit** representa un desafío técnico permanente. La elección de *Huber Loss* moderó este efecto, pero no lo eliminó completamente, como se evidencia en la dispersión de residuos para valores extremos.

Por último, el diseño del espacio de features requirió decisiones no triviales: la codificación cíclica del mes es necesaria para capturar estacionalidad, pero la explosión de dimensionalidad generada por el *One-Hot Encoding* de **YearMonth** requiere validar que el modelo no memorice patrones temporales específicos del entrenamiento.

Conclusiones y Aprendizajes Obtenidos

El MLP implementado demostró ser una solución eficaz para la predicción de **Profit**, alcanzando un R^2 de 0.9222 con una arquitectura relativamente simple de dos capas ocultas (64 y 32 neuronas). Esto evidencia que no es necesaria una red profunda ni costosa computacionalmente para capturar las relaciones no lineales presentes en datos tabulares.

Sin embargo, una limitación relevante de los MLP es su **falta de interpretabilidad**: si bien el modelo predice con precisión, no permite entender de forma directa qué variables impulsan el **Profit** ni cómo interactúan entre sí. En un contexto de negocio, esta opacidad puede ser un obstáculo cuando se requiere justificar decisiones ante *stakeholders* no técnicos.

Por ello, como alternativa a considerar, los modelos basados en árboles de decisión —como *Random Forest* o *Gradient Boosting* (XGBoost, LightGBM)— ofrecen un nivel de interpretabilidad nativo significativamente mayor, a través de métricas de importancia de variables y estructuras de decisión auditables, manteniendo al mismo tiempo un desempeño competitivo en datos tabulares.